

Experiment 1: Database Constraints, Relationships, and Security Control

Student Information

- Name: Aryan Saini
- UID: 24BAI70427
- Course: B.E. CSE-H (AIML)
- Subject: Database Management Systems

1. Aim of the Session

The objective of this session is to implement Relational Schemas using advanced integrity constraints (Primary Key, Foreign Key, Check, and Unique) and to manage database security through Data Control Language (DCL) operations like GRANT and REVOKE.

2. Objectives

By the conclusion of this practical, the following goals were achieved:

- Established relational dependencies between tables using Foreign Keys.
- Implemented domain integrity using the CHECK constraint and entity integrity via UNIQUE and NOT NULL descriptors.
- Mastered Role-Based Access Control (RBAC) by creating database roles and managing user permissions.
- Understood the process of schema validation through structured data insertion and verification.

3. Procedure

The experiment followed a logical sequence of database administration tasks:

1. Schema Definition: Created the BOOKS and LIBRARY_VISITORS tables to serve as parent entities.
2. Constraint Application: Integrated a CHECK constraint on the AGE column to ensure visitors are at least 17 years old and a UNIQUE constraint on EMAIL.
3. Relationship Mapping: Constructed the BOOK_ISSUE table as a child entity, linking it to both visitors and books via Foreign Key references.
4. Data Ingestion: Populated the relations with sample records to test constraint validation.

5. Security Administration: Initialized a new database role named LIBRARIAN with secure login credentials.
6. Permission Management: Demonstrated the administrative capability to GRANT specific DML privileges to a role and subsequently REVOKE them to ensure data security.

4. I/O Analysis (Input / Output)

SQL Input Logic:

The implementation involved defining the relational structure and managing access control:

SQL

```
CREATE TABLE BOOKS(  
    BOOK_ID INT PRIMARY KEY,  
    BOOK_NAME VARCHAR(20) NOT NULL,  
    AUTHOR_NAME VARCHAR(20) NOT NULL  
)
```

```
SELECT * FROM BOOKS
```

```
SELECT * FROM BOOKS
```

```
INSERT INTO BOOKS VALUES(1, 'HARRY POTTER', 'DAVID')
```

```
CREATE TABLE LIBRARY_VISITORS(  
    USER_ID INT PRIMARY KEY,  
    NAME VARCHAR(20) NOT NULL,  
    AGE INT CHECK(AGE>=17) NOT NULL,  
    EMAIL VARCHAR(20) NOT NULL UNIQUE  
)
```

```
SELECT * FROM LIBRARY_VISITORS
```

```
INSERT INTO LIBRARY_VISITORS(USER_ID,NAME,AGE,EMAIL)  
VALUES(501, 'Ansh Sharma', 20, 'ansh21@gmail.com')
```

```
INSERT INTO LIBRARY_VISITORS VALUES(502, 'Nitin', 19, 'nitin2@gmail.com')
```

```
CREATE TABLE BOOK_ISSUE(  
    BOOK_ISSUE_ID INT PRIMARY KEY,  
    USER_ID INT REFERENCES LIBRARY_VISITORS(USER_ID),  
    BOOK_ID INT NOT NULL,  
    FOREIGN KEY(BOOK_ID) REFERENCES BOOKS(BOOK_ID)  
)
```

```
SELECT * FROM BOOK_ISSUE
```

```
INSERT INTO BOOK_ISSUE VALUES(101, 501, 1)
```

```
CREATE ROLE LIBRARIAN  
WITH LOGIN PASSWORD 'Aryan@25'
```

```
SELECT CURRENT_USER
```

```
GRANT SELECT,INSERT,DELETE,UPDATE ON BOOKS TO LIBRARIAN  
REVOKE SELECT,INSERT,DELETE,UPDATE ON BOOKS FROM LIBRARIAN  
SELECT * FROM BOOKS  
SELECT * FROM LIBRARY_VISITORS
```

```
SELECT * FROM BOOK_ISSUE
```

```
22 SELECT * FROM LIBRARY_VISITORS
23
```

Data Output Messages Notifications				
	user_id [PK] integer	name character varying (20)	age integer	email character varying (20)
1	501	Ansh Sharma	20	ansh21@gmail.com
2	502	Nitin	19	nitin2@gmail.com

```
36 SELECT * FROM BOOK_ISSUE
37
38 INSERT INTO BOOK_ISSUE VALUES(101, 501, 1)
39
```

Data Output Messages Notifications				
	book_issue_id [PK] integer	user_id integer	book_id integer	issue_date date
1	101	501	1	2026-01-09

48

49

50

SELECT * FROM BOOKS

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	book_id [PK] integer	book_name character varying (20)	author_name character varying (20)	book_count integer
1	1	HARRY POTTER	DAVID	2

Output Analysis:

- **Referential Integrity:** The BOOK_ISSUE table successfully maintained links between users and books, preventing the issuance of non-existent items.
- **Data Validation:** The system enforced the AGE >= 17 rule, ensuring that only eligible users were added to the LIBRARY_VISITORS table.
- **Access Control:** The GRANT command successfully expanded the LIBRARIAN role's capabilities, while the REVOKE command effectively restricted them, demonstrating precise security management.

5. Learning Outcomes

Through this session, I have developed the following core competencies:

- **Relational Design:** Gained the ability to create interconnected tables that maintain data consistency across a database.
- **Constraint Logic:** Understood how to use automated database rules to prevent invalid data entry.
- **Security Governance:** Developed practical skills in managing user roles and permissions, a critical aspect of professional database administration.